

電腦網路 Computer Network

Wireshark and Socket

Department of Computer Science and Engineering,
National Sun Yat-sen University

Asst. Prof. ShengMin(Sam) Chiu

Overview

- Conceptual & implementation aspects of application-layer protocols
 - transport-layer service models
 - client-server paradigm
 - peer-to-peer paradigm
- Programming network applications
 - Socket programming with UDP and TCP
- Wireshark(packet sniffing)

Some network apps

- social networking
 - Web
 - text messaging
 - e-mail
 - multi-user network games
 - streaming stored video (YouTube, Hulu, Netflix)
 - P2P file sharing
 - voice over IP (e.g., Skype)
 - real-time video conferencing
 - Internet search
 - remote login
 - ...
- Q: *your* favorites?

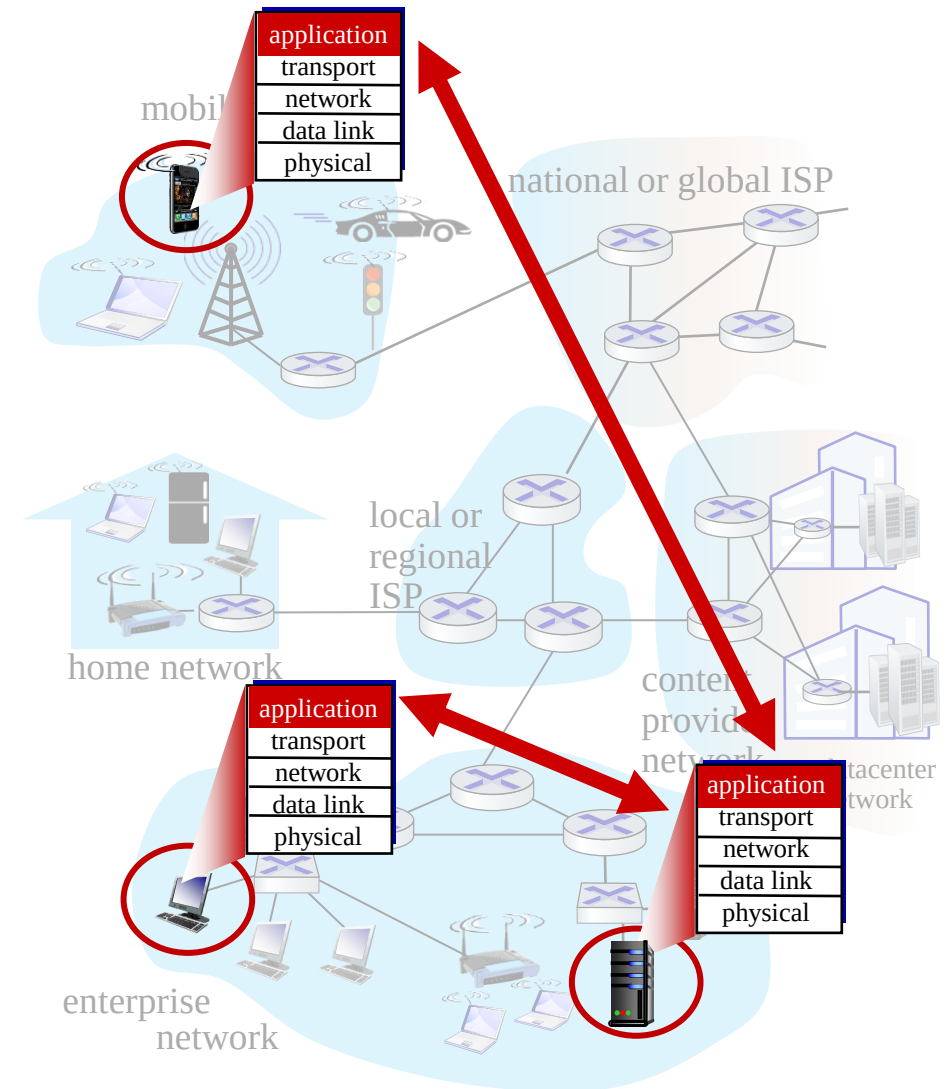
Creating a network app

write programs that:

- run on (different) end systems
- communicate over network
- e.g., web server software communicates with browser software

no need to write software for network-core devices

- network-core devices do not run user applications
- applications on end systems allows for rapid app development, propagation



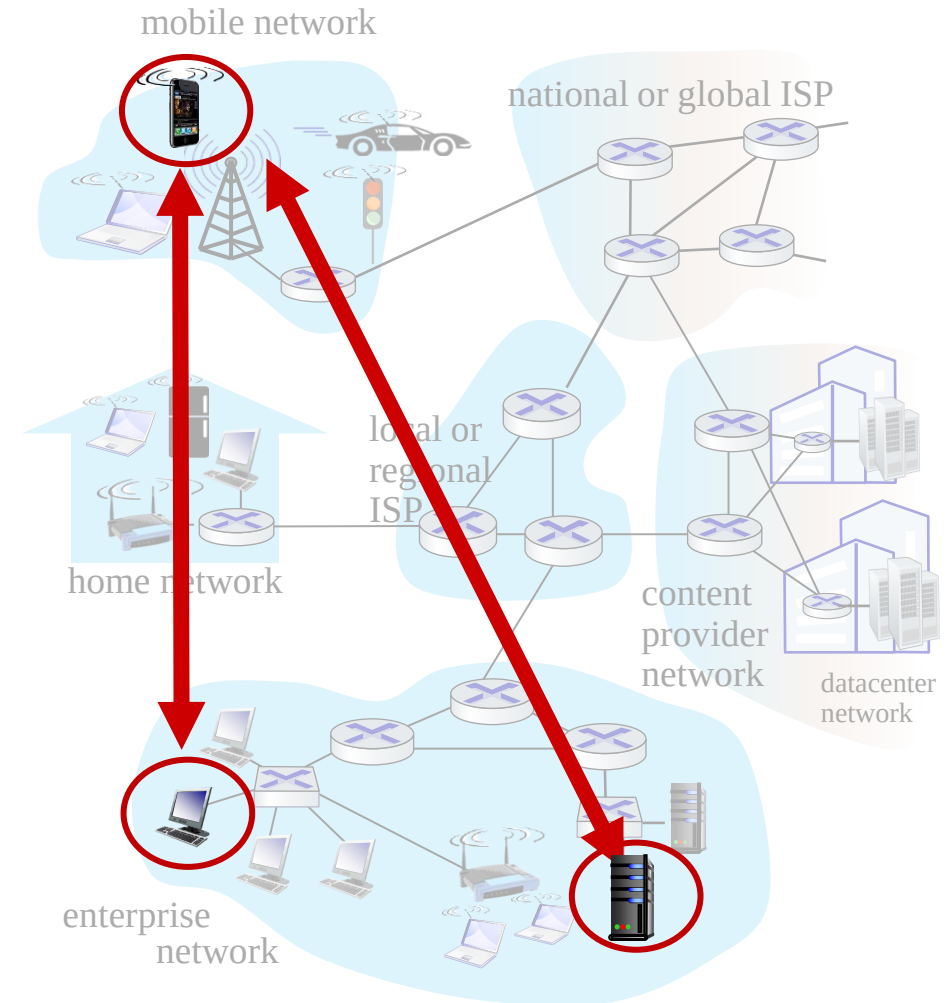
Client-server paradigm

server:

- always-on host
- permanent IP address
- often in data centers, for scaling

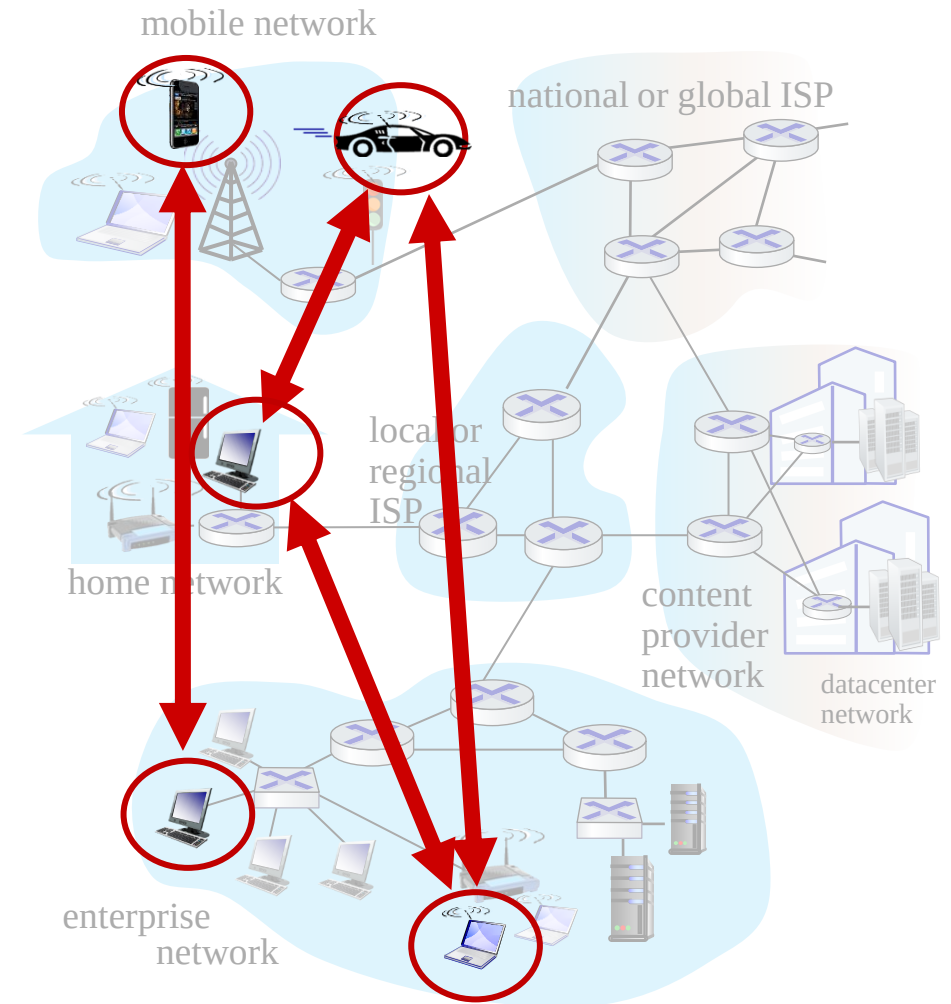
clients:

- contact, communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do *not* communicate directly with each other
- examples: HTTP, IMAP, FTP



Peer-peer architecture

- **no** always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
 - *self scalability* – new peers bring new service capacity, as well as new service demands
- peers are intermittently connected and change IP addresses
 - complex management
- example: P2P file sharing



Processes communicating

process: program running within a host

- within same host, two processes communicate using **inter-process communication** (defined by OS)
- processes in different hosts communicate by exchanging **messages**

clients, servers

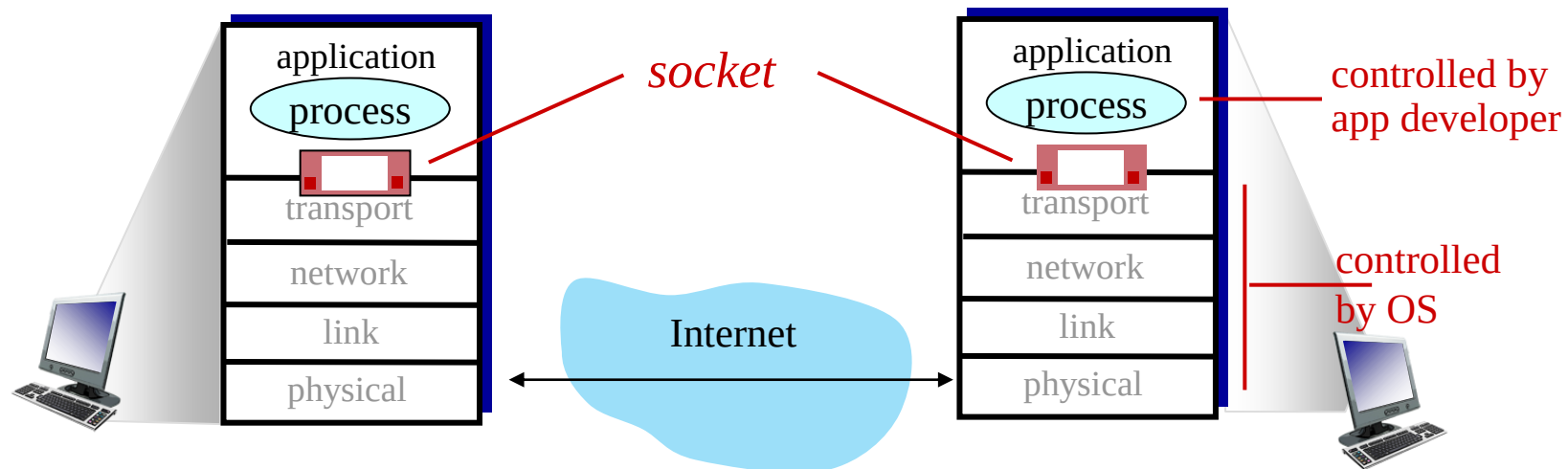
client process: process that initiates communication

server process: process that waits to be contacted

- note: applications with P2P architectures have client processes & server processes

Sockets

- process sends/receives messages to/from its **socket**
- socket analogous to door
 - sending process shoves message out door
 - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process
 - two sockets involved: one on each side



Addressing processes

- to receive messages, process must have *identifier*
- host device has unique 32-bit IP address
- Q: does IP address of host on which process runs suffice for identifying the process?
- A:?
- *identifier* includes both **IP address** and **port numbers** associated with process on host.
- example port numbers:
 - HTTP server: 80
 - mail server: 25
- to send HTTP message to gaia.cs.umass.edu web server:
 - **IP address**: 128.119.245.12
 - **port number**: 80

An application-layer protocol defines:

- types of messages exchanged,
 - e.g., request, response
- message syntax:
 - what fields in messages & how fields are delineated
- message semantics
 - meaning of information in fields
- rules for when and how processes send & respond to messages

open protocols:

- defined in RFCs, everyone has access to protocol definition
- allows for interoperability
- e.g., HTTP, SMTP

proprietary protocols:

- e.g., Skype

What transport service does an app need?

Data Integrity

- some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- other apps (e.g., audio) can tolerate some loss

Timing

- some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

Throughput

- some apps (e.g., multimedia) require minimum amount of throughput to be “effective”
- other apps (“elastic apps”) make use of whatever throughput they get

Security

- encryption, data integrity, ...

Transport service requirements: common apps

application	data loss	throughput	time sensitive?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video: 10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

Internet transport protocols services

Transmission Control Protocol

TCP service:

- *reliable transport* between sending and receiving process
- *flow control*: sender won't overwhelm receiver
- *congestion control*: throttle sender when network overloaded
- *does not provide*: timing, minimum throughput guarantee, security
- *connection-oriented*: setup required between client and server processes

User Datagram Protocol

UDP service:

- *unreliable data transfer* between sending and receiving process
- *does not provide*: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup.

Q: Why bother?

Why is there a UDP?

Internet transport protocols services

application	application layer protocol	transport protocol
file transfer/download	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP 1.1 [RFC 7320]	TCP
Internet telephony	SIP [RFC 3261], RTP[RFC 3550], or proprietary	TCP or UDP
streaming audio/video	HTTP [RFC 7320], DASH	TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

Securing TCP

Vanilla TCP & UDP sockets:

- no encryption
- cleartext passwords sent into socket traverse Internet in cleartext (!)

Transport Layer Security (TLS)

- provides encrypted TCP connections
- data integrity
- end-point authentication

TSL implemented in application layer

- apps use TSL libraries, that use TCP in turn

TLS socket API

- cleartext sent into socket traverse Internet *encrypted*
- see Chapter 8

Classwork 1

- What information is used by a process running on one host to identify a process running on another host?
- 在一台主機上執行的程序使用哪些資訊來識別在另一台主機上執行的程序？

Overview

- Conceptual & implementation aspects of application-layer protocols
 - transport-layer service models
 - client-server paradigm
 - peer-to-peer paradigm
- Programming network applications
 - Socket programming with UDP and TCP
- Wireshark(packet sniffing)

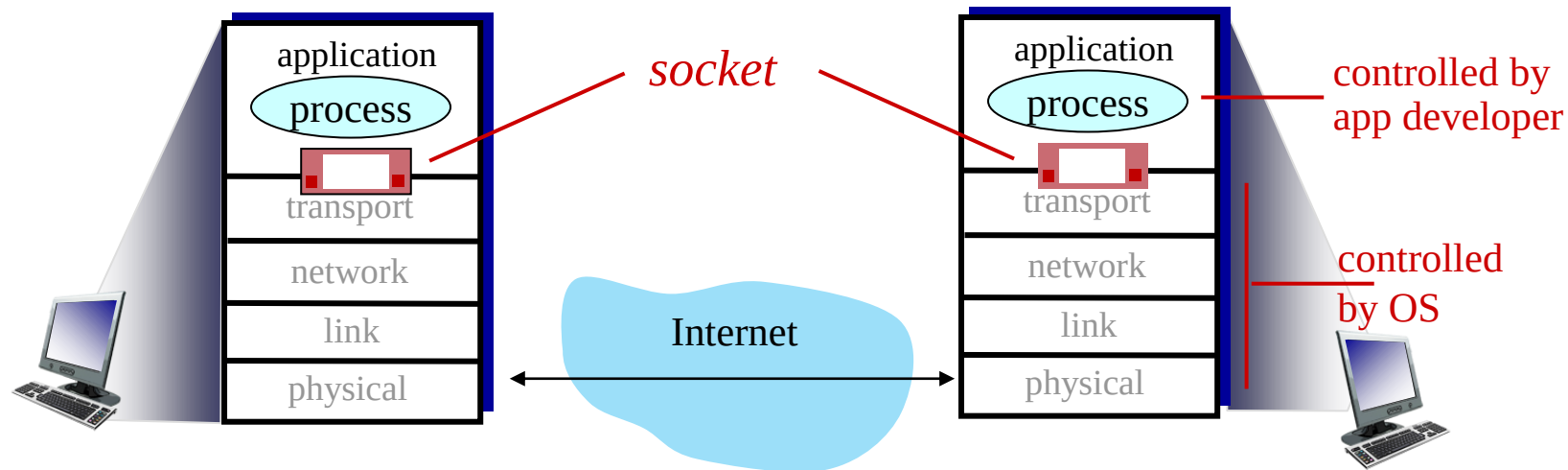
Socket programming

goal:

learn how to build client/server applications that communicate using sockets

socket:

door between application process and end-end-transport protocol



Socket programming

Two socket types for two transport services:

- **User Datagram Protocol(UDP):** unreliable datagram
- **Transmission Control Protocol(TCP):** reliable, byte stream-oriented

Application Example:

1. Client reads a line of characters (data) from its keyboard and sends data to server
2. Server receives the data and converts characters to uppercase
3. Server sends modified data to client
4. Client receives modified data and displays line on its screen

Socket programming with UDP

UDP: no “connection” between client & server

- no ***handshaking*** before sending data
- sender explicitly attaches IP destination address and port# to each packet
- receiver extracts sender IP address and port# from received packet

UDP: transmitted data may be lost or received out-of-order

Application viewpoint:

- UDP provides ***unreliable*** transfer of groups of bytes (“datagrams”) between client and server

Client/server socket interaction: UDP



server (running on serverIP)

create socket, port = x:
`serverSocket =
socket(AF_INET, SOCK_DGRAM)`

read datagram from
`serverSocket`

write reply to
`serverSocket`
specifying
client address,
port number

client



create socket:
`clientSocket =
socket(AF_INET, SOCK_DGRAM)`

Create datagram with serverIP and
Port = x; send datagram via
`clientSocket`

read datagram from
`clientSocket`

close
`clientSocket`

Example app: UDP client

Python UDPClient

include Python's socket library	→	from socket import *
		serverName = '127.0.0.1' #hostname
		serverPort = 12000
create UDP socket for server	→	clientSocket = socket(AF_INET, SOCK_DGRAM)
get user keyboard input	→	message = input('Input lowercase sentence:')
attach server name, port to message; send into socket	→	clientSocket.sendto(message.encode(), (serverName, serverPort))
read reply characters from socket into string	→	modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
print out received string and close socket	→	print modifiedMessage.decode() clientSocket.close()

Example app: UDP server

Python UDPServer

	from socket import *
	serverPort = 12000
create UDP socket	→ serverSocket = socket(AF_INET, SOCK_DGRAM)
bind socket to local port number 12000	→ serverSocket.bind(("", serverPort))
	print("The server is ready to receive")
loop forever	→ while True:
Read from UDP socket into message, getting client's address (client IP and port)	→ message, clientAddress = serverSocket.recvfrom(2048)
	modifiedMessage = message.decode().upper()
send upper case string back to this client	→ serverSocket.sendto(modifiedMessage.encode(), clientAddress)

Socket programming with TCP

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

Client contacts server by:

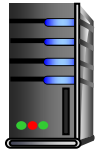
- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket*: client TCP establishes connection to server TCP

- when contacted by client, *server TCP creates new socket* for server process to communicate with that particular client
 - allows server to talk with multiple clients
 - source port numbers used to distinguish clients (more in Ch3)

Application viewpoint

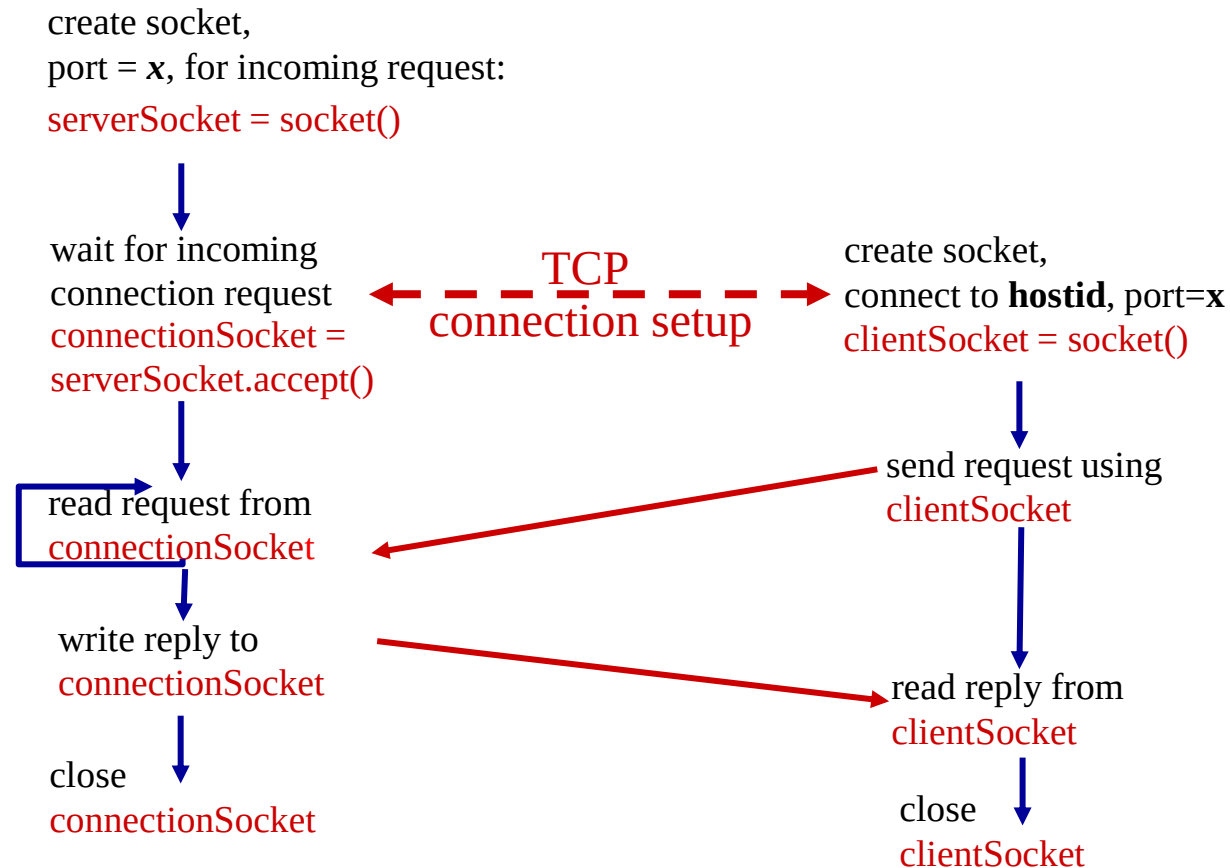
TCP provides reliable, in-order byte-stream transfer (“pipe”) between client and server

Client/server socket interaction: TCP



server (running on `hostid`)

client



Example app: TCP client

Python TCPClient

create TCP socket for server,
remote port 12000

No need to attach server name, port

```
from socket import *
serverName = '127.0.0.1' #servername
serverPort = 12000
→ clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = raw_input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
→ modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

Example app: TCP server

Python TCPServer

	→	from socket import *
		serverPort = 12000
create TCP welcoming socket	→	serverSocket = socket(AF_INET, SOCK_STREAM)
		serverSocket.bind(('', serverPort))
server begins listening for incoming TCP requests	→	serverSocket.listen(1)
		print ('The server is ready to receive')
loop forever	→	while True:
server waits on accept() for incoming requests, new socket created on return	→	connectionSocket, addr = serverSocket.accept()
read bytes from socket (but not address as in UDP)	→	sentence = connectionSocket.recv(1024).decode()
		capitalizedSentence = sentence.upper()
		connectionSocket.send(capitalizedSentence.encode())
close connection to this client (but <i>not</i> welcoming socket)	→	connectionSocket.close()

TCP & UDP

TCP: w/ handshaking

- reliable byte **stream**
- Like a pipe (水管)

UDP: w/o handshaking

- unreliable message **delivery**
- Like an envelope (信封)

Example:

- send("Hello")
- send("World")

TCP:

recv() → "HelloWorld"

----- or -----

recv() → "Hel"

recv() → "loWorld"

----- or -----

...

UDP:

recv() → "Hello"

recv() → "World"

UDP: User Datagram Protocol [RFC 768]

INTERNET STANDARD

RFC 768

J. Postel
ISI
28 August 1980

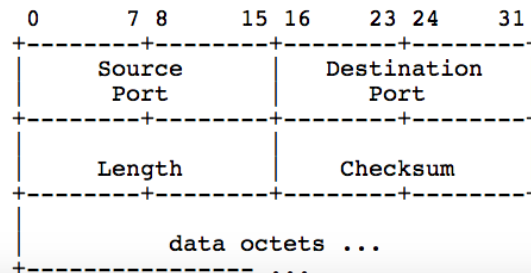
User Datagram Protocol

Introduction

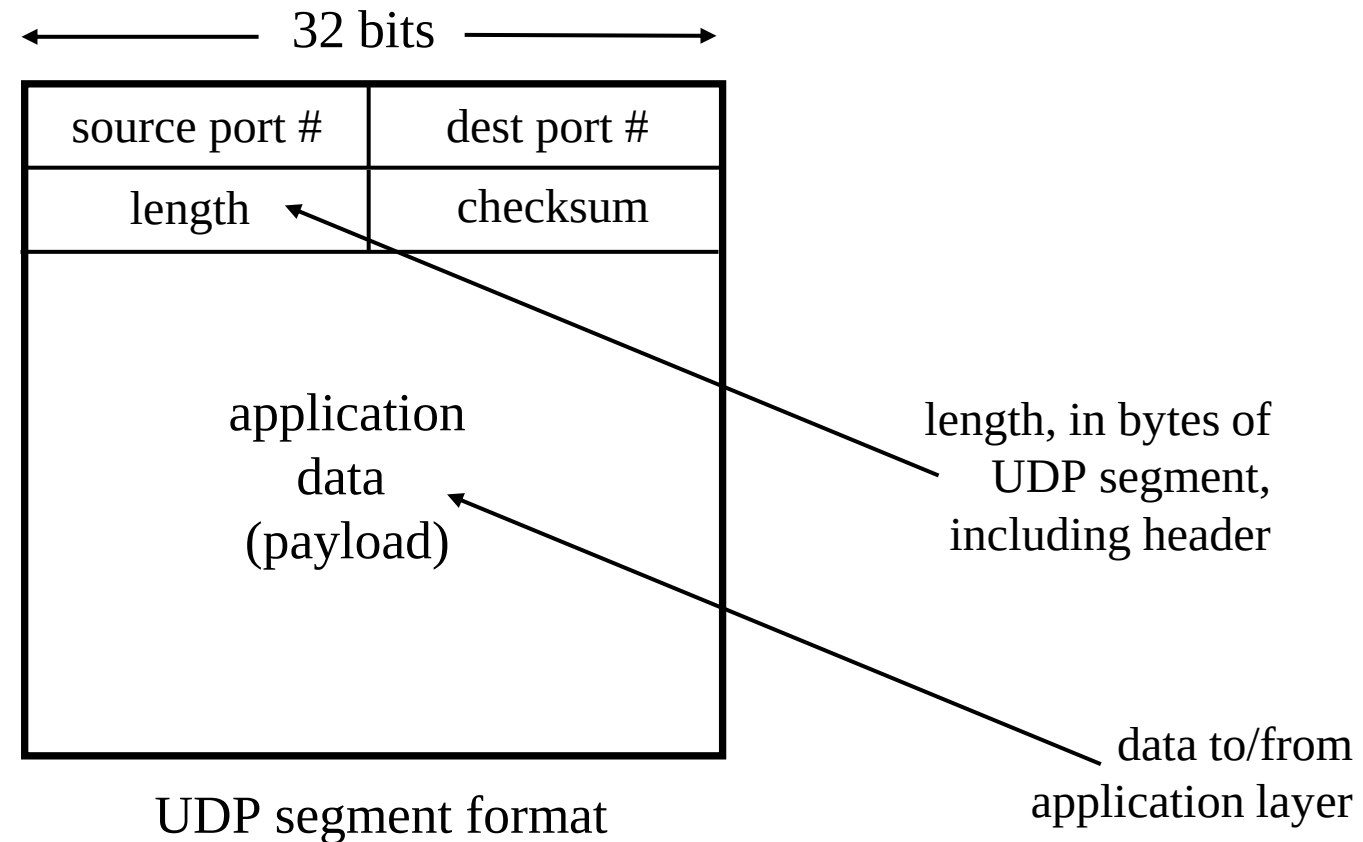
This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

Format

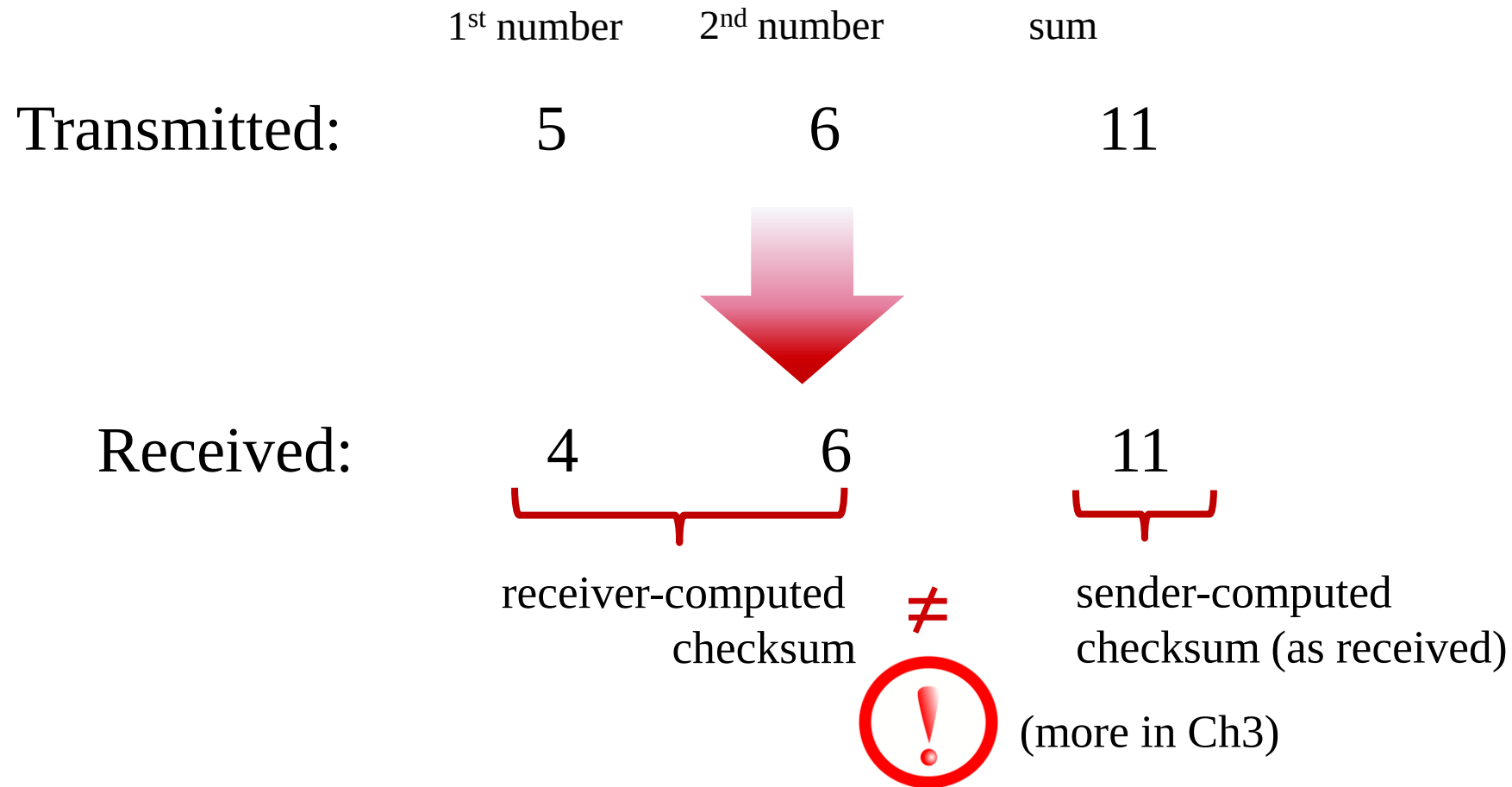


UDP segment header

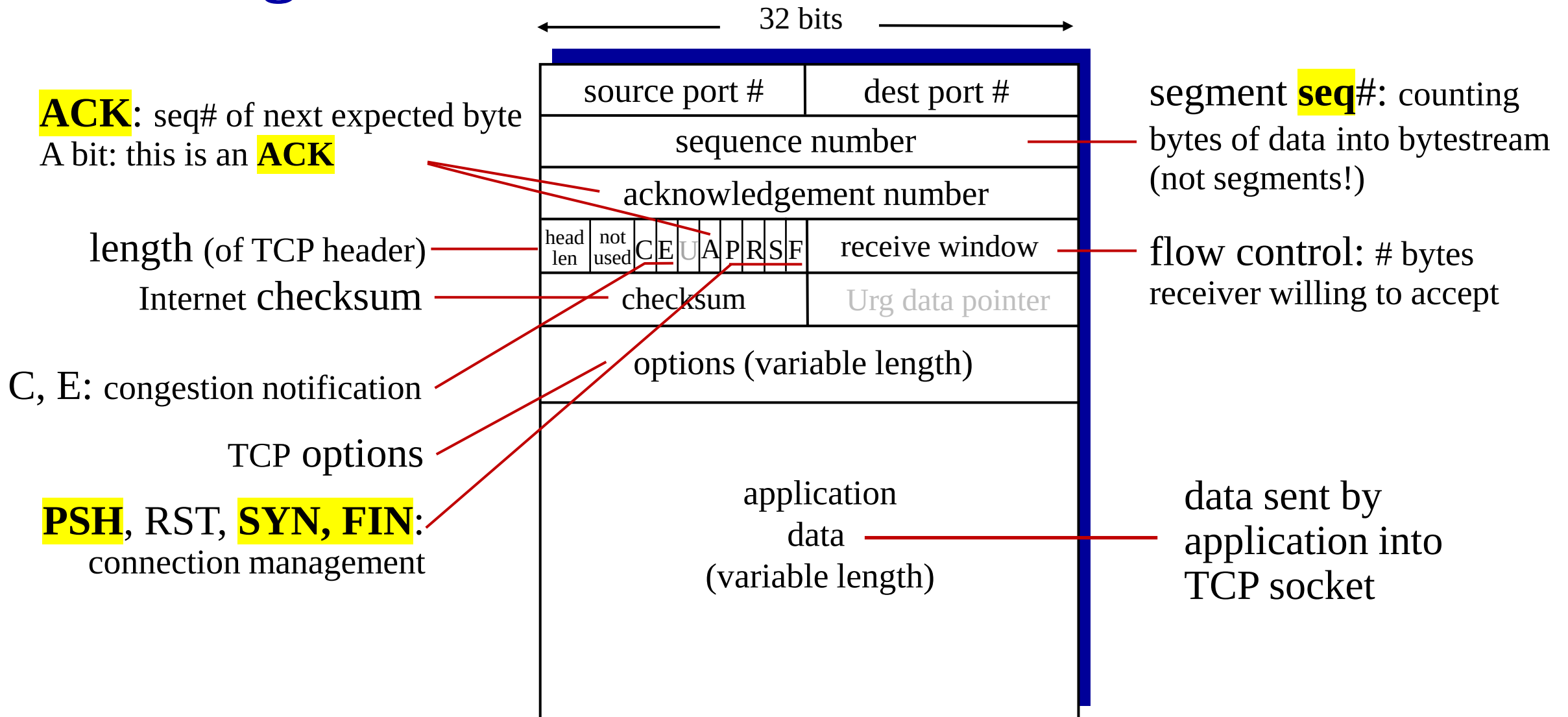


UDP checksum

Goal: detect errors (*i.e.*, flipped bits) in transmitted segment



TCP segment structure



TCP sequence numbers, ACKs

Sequence numbers:

- byte stream “number” of first byte in segment’s data

Acknowledgements:

- seq # of next byte expected from other side
- cumulative ACK

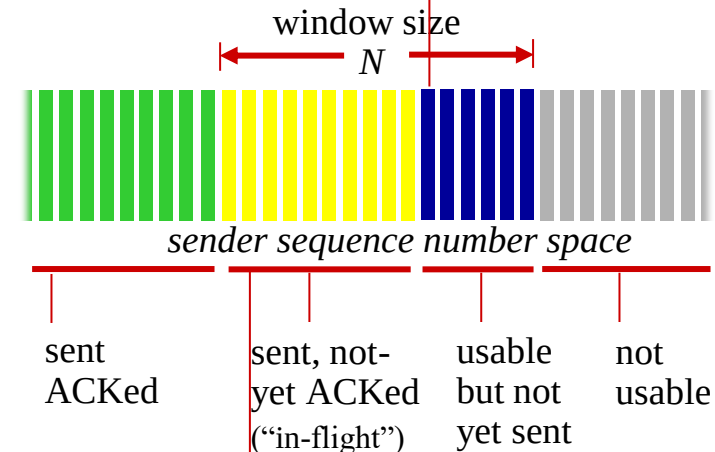
Q: how receiver handles out-of-order segments

A: TCP spec doesn’t say, up to implementor

outgoing segment from sender

source port #	dest port #
sequence number	
acknowledgement number	
	rwnd
checksum	urg pointer

Seq: segment內第一個byte在整個stream內的編號

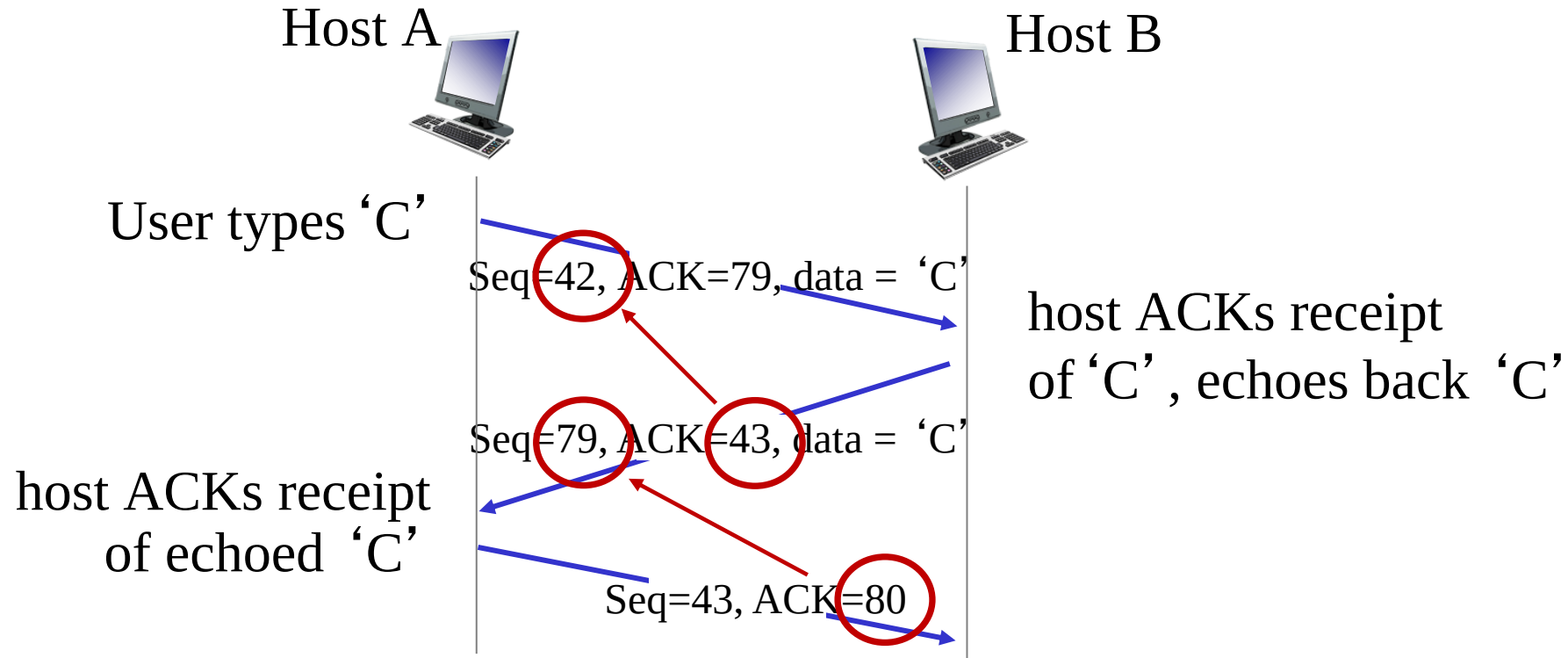


outgoing segment from receiver

source port #	dest port #
sequence number	
acknowledgement number	
	rwnd
checksum	urg pointer

ACK: 已經收到所有小於此數字的byte，預計收到下一個byte的編號

TCP sequence numbers, ACKs



simple telnet scenario

A human 3-way handshake protocol



Q: Server and Client?

A:

Client → estab. connection

Server → recv. okay

Client → I know you recv.

Client → SYN

Server → SYN+ACK

Client → ACK

TCP 3-way handshake

Client state

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

LISTEN

```
clientSocket.connect((serverName,serverPort))
```

SYNSENT

ESTAB

choose init seq num x
send TCP SYN msg

received SYNACK(x)
indicates server is live;
send ACK for SYNACK;
this segment may contain
client-to-server data



SYNbit=1, Seq= x

SYNbit=1, Seq= y
ACKbit=1; ACKnum= $x+1$

ACKbit=1, ACKnum= $y+1$

choose init seq num y
send TCP SYNACK
msg, acking SYN

received ACK(y)
indicates client is live

Server state

```
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
connectionSocket, addr = serverSocket.accept()
```

LISTEN

SYN RCVD

ESTAB

Closing a TCP connection

- Client, server each close their side of connection
 - send TCP segment with FIN bit = 1
- Respond to received FIN with ACK
 - on receiving FIN, ACK can be combined with own FIN
- Simultaneous FIN exchanges can be handled

Classwork 2

- What is meant by a handshaking protocol?
- 請問握手協定(handshaking protocol)的用意為何？

How about
- Seq#?
- ACK#?

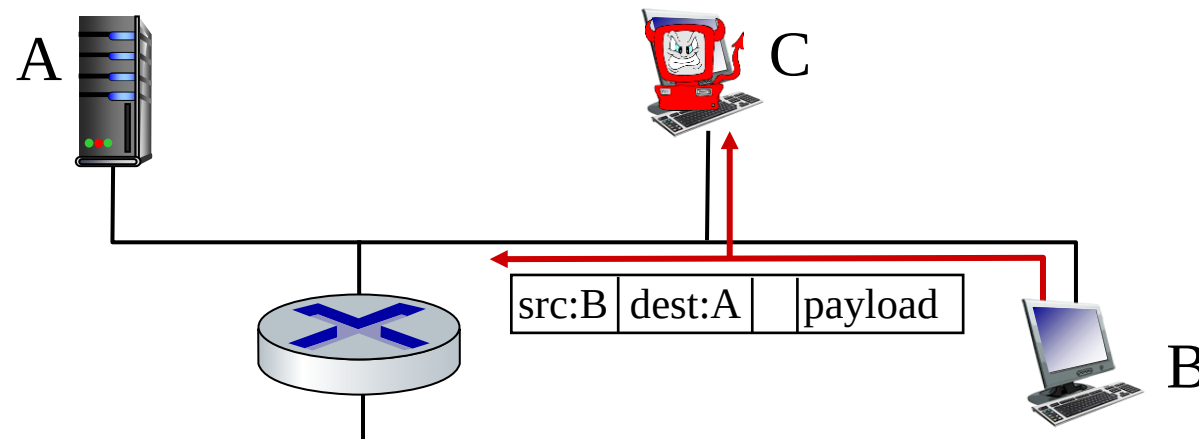
Overview

- Conceptual & implementation aspects of application-layer protocols
 - transport-layer service models
 - client-server paradigm
 - peer-to-peer paradigm
- Programming network applications
 - Socket programming with UDP and TCP
- Wireshark(packet sniffing)

Bad guys: packet interception

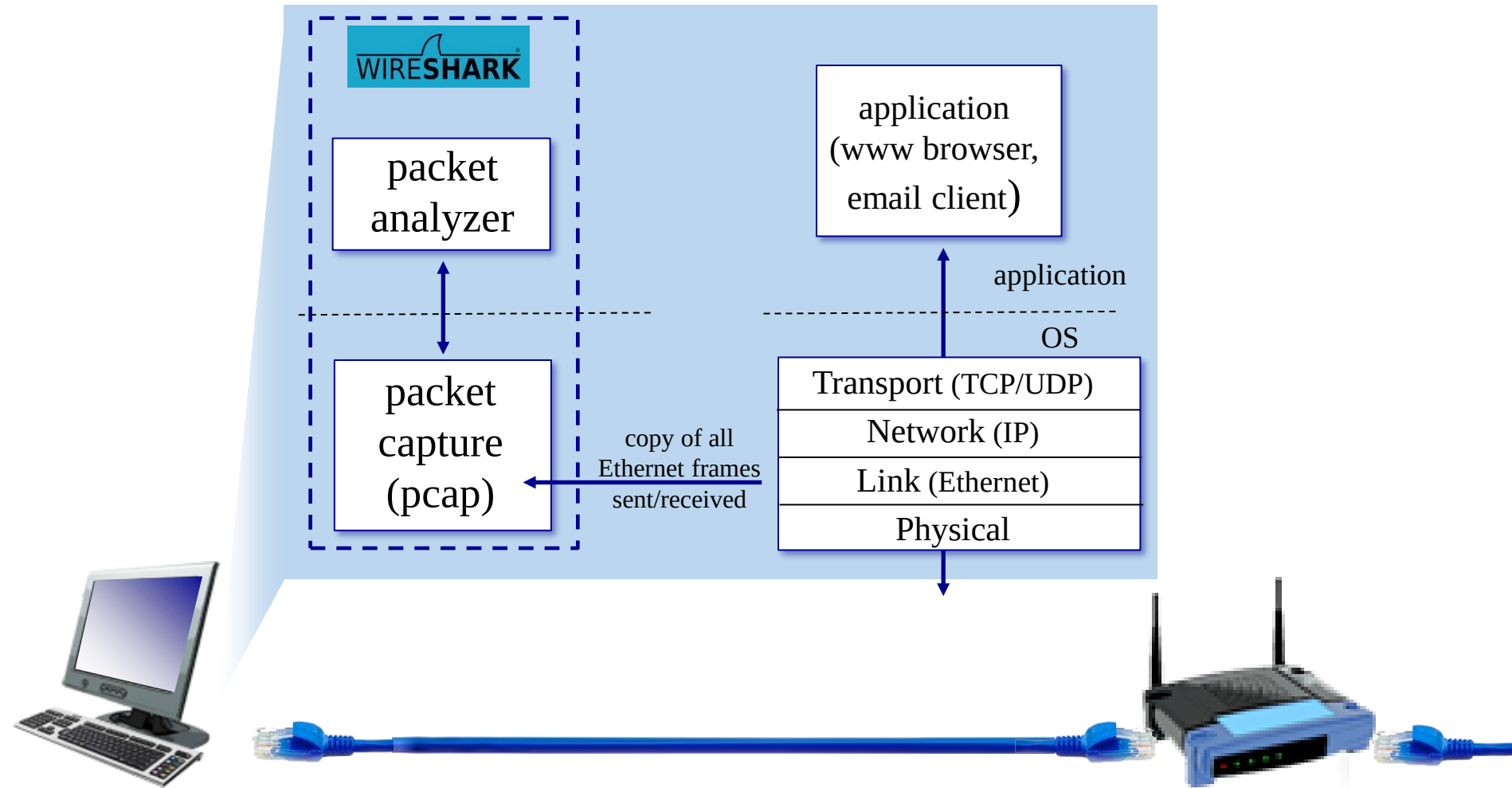
packet “sniffing”:

- broadcast media (shared Ethernet, wireless)
- promiscuous network interface reads/records all packets passing by (including passwords!)



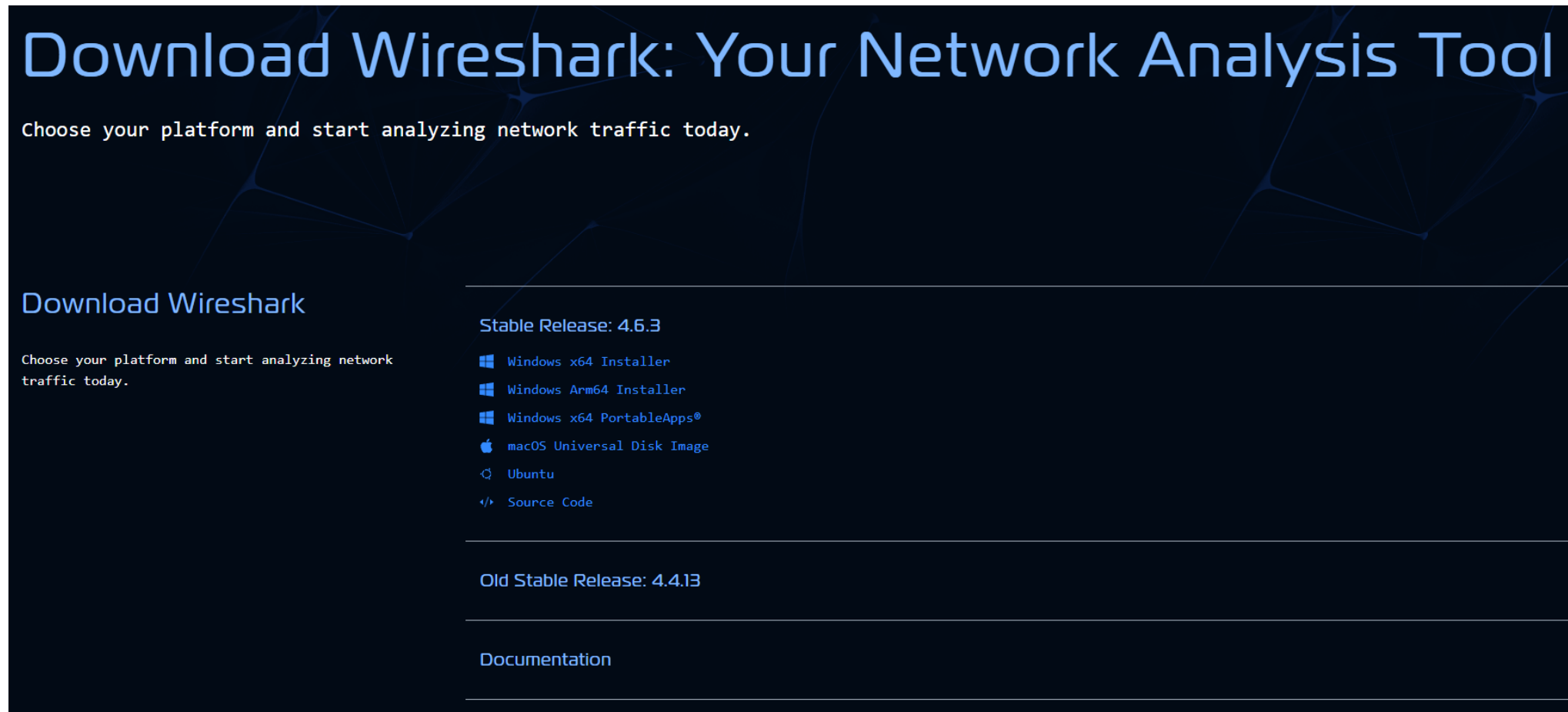
Wireshark software used for our end-of-chapter labs is a (free) packet-sniffer

Wireshark



Wireshark

- <https://www.wireshark.org/download.html>

A screenshot of the Wireshark download page. The page has a dark blue background with a network diagram pattern. The main heading is "Download Wireshark: Your Network Analysis Tool" in a light blue font. Below it, a subtitle says "Choose your platform and start analyzing network traffic today." in a monospaced font. On the left, there is a section titled "Download Wireshark" with the same subtitle. On the right, there are three sections: "Stable Release: 4.6.3" with a list of download links for Windows (x64 Installer, Arm64 Installer, x64 PortableApps®), macOS (Universal Disk Image), Ubuntu, and Source Code; "Old Stable Release: 4.4.13"; and "Documentation".

Download Wireshark: Your Network Analysis Tool

Choose your platform and start analyzing network traffic today.

Download Wireshark

Choose your platform and start analyzing network traffic today.

Stable Release: 4.6.3

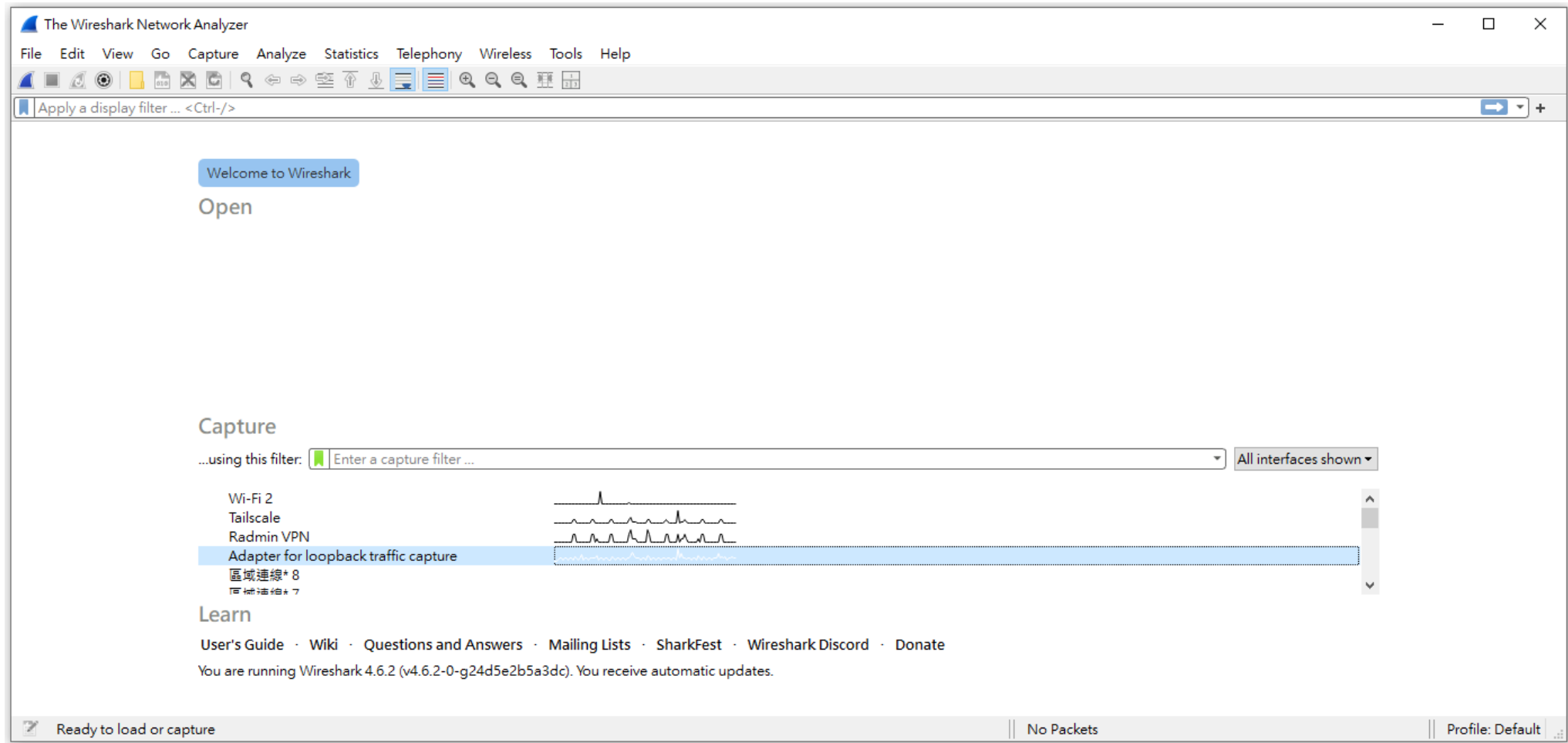
- Windows x64 Installer
- Windows Arm64 Installer
- Windows x64 PortableApps®
- macOS Universal Disk Image
- Ubuntu
- Source Code

Old Stable Release: 4.4.13

Documentation

Network Interface Card (NIC)

■ 網路介面卡



Loopback Interface

- 回送介面
- <https://wiki.wireshark.org/CaptureSetup/Loopback>

Loopback capture setup

The following will explain capturing on loopback interfaces a bit.

If you are trying to capture traffic from a machine to itself, that traffic will not be sent over a real network interface, even if it's being sent to an address on one of the machine's network adapters. This means that you will not see it if you are trying to capture on, for example, the interface device for the adapter to which the destination address is assigned. You will only see it if you capture on the "loopback interface", if there is such an interface and it is possible to capture on it; see the next section for information on the platforms on which you can capture on the "loopback interface".

Loopback Interface

Usually called

- Windows : Adapter for loopback traffic capture
- Linux/macOS : lo

IP 127.0.0.1

Windows

You can't capture on the local loopback address 127.0.0.1 with [WinPcap](#). The following page from "Windows network services internals" explains why: [The missing network loopback interface](#).

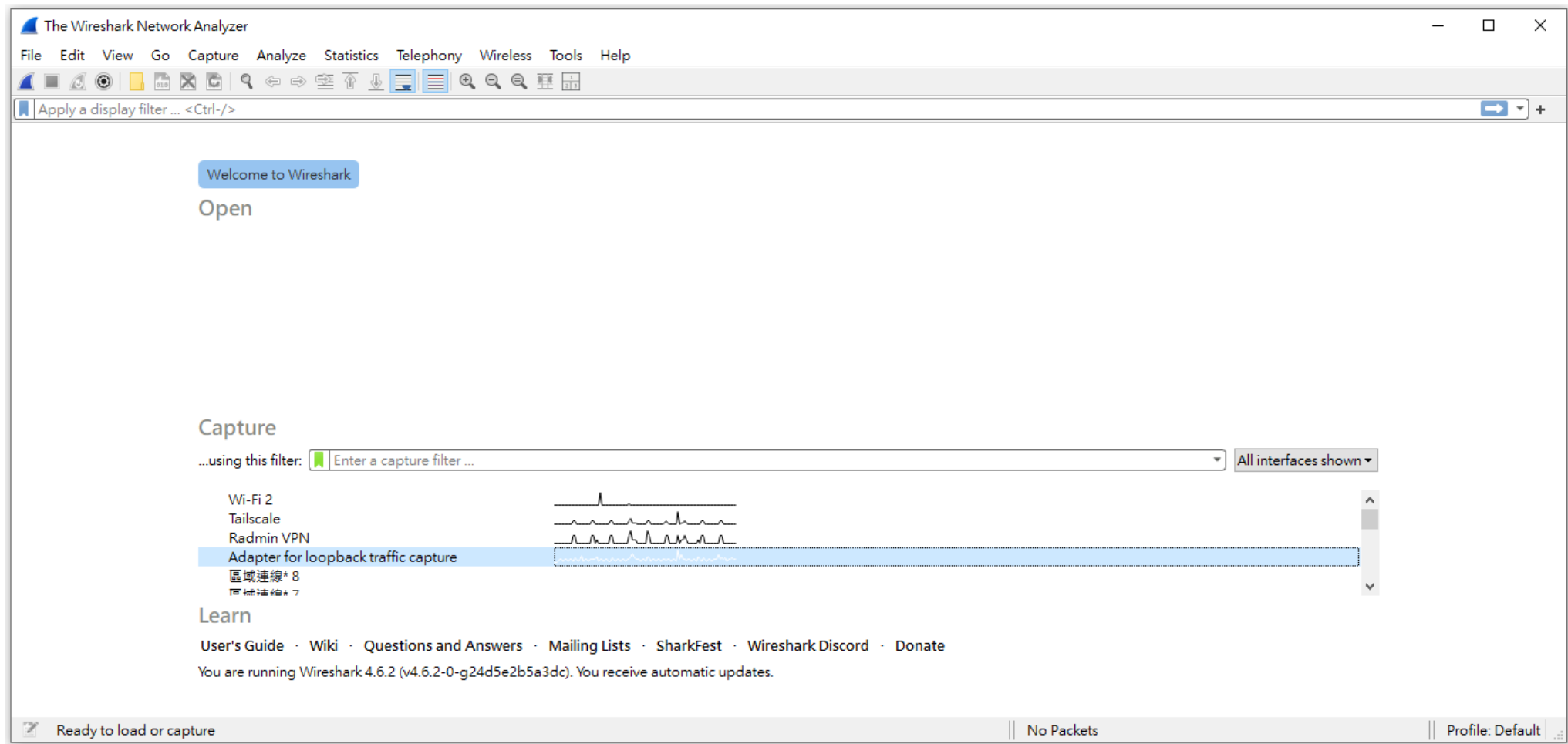
Starting from Windows 7: Npcap

[Npcap](#) is an update of [WinPcap](#) using NDIS 6 Light-Weight Filter (LWF), done by Yang Luo for Nmap project during Google Summer of Code 2013 and 2015. Npcap adds several new features to those existing in WinPcap, including loopback traffic capture.

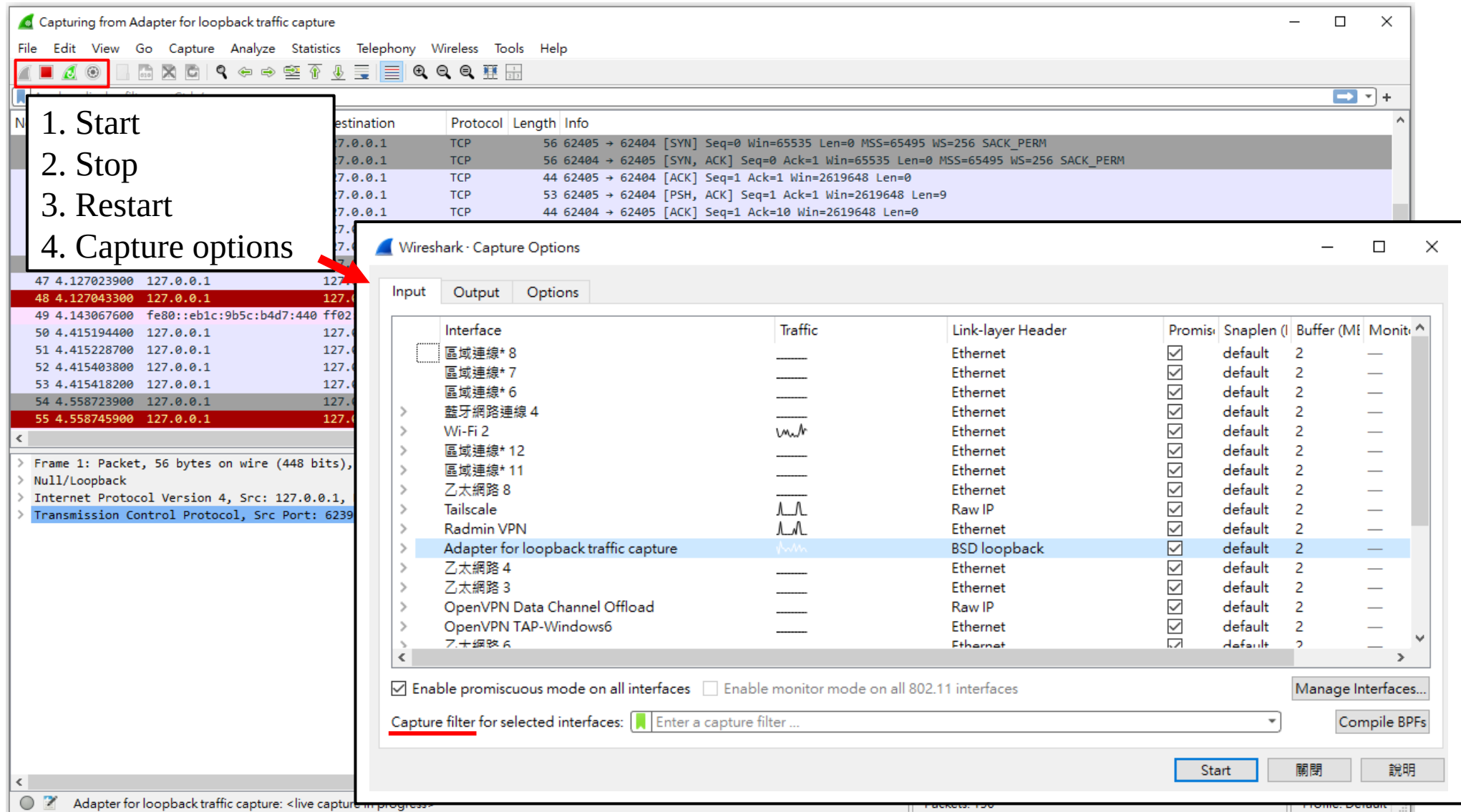
In the [list of capture interfaces](#), select "Adapter for loopback traffic capture" and begin capturing as usual. The [data link type](#) for this adapter is `DLT_NULL`.

Network Interface Card (NIC)

■ 網路介面卡



User Interface



1. Start
2. Stop
3. Restart
4. Capture options

Wireshark · Capture Options

Input	Output	Options
Interface	Traffic	Link-layer Header
區域連線* 8	---	Ethernet
區域連線* 7	---	Ethernet
區域連線* 6	---	Ethernet
藍牙網路連線 4	---	Ethernet
Wi-Fi 2	---	Ethernet
區域連線* 12	---	Ethernet
區域連線* 11	---	Ethernet
乙太網路 8	---	Ethernet
Tailscale	---	Raw IP
Radmin VPN	---	Ethernet
Adapter for loopback traffic capture	---	BSD loopback
乙太網路 4	---	Ethernet
乙太網路 3	---	Ethernet
OpenVPN Data Channel Offload	---	Raw IP
OpenVPN TAP-Windows6	---	Ethernet
乙太網路 6	---	Ethernet

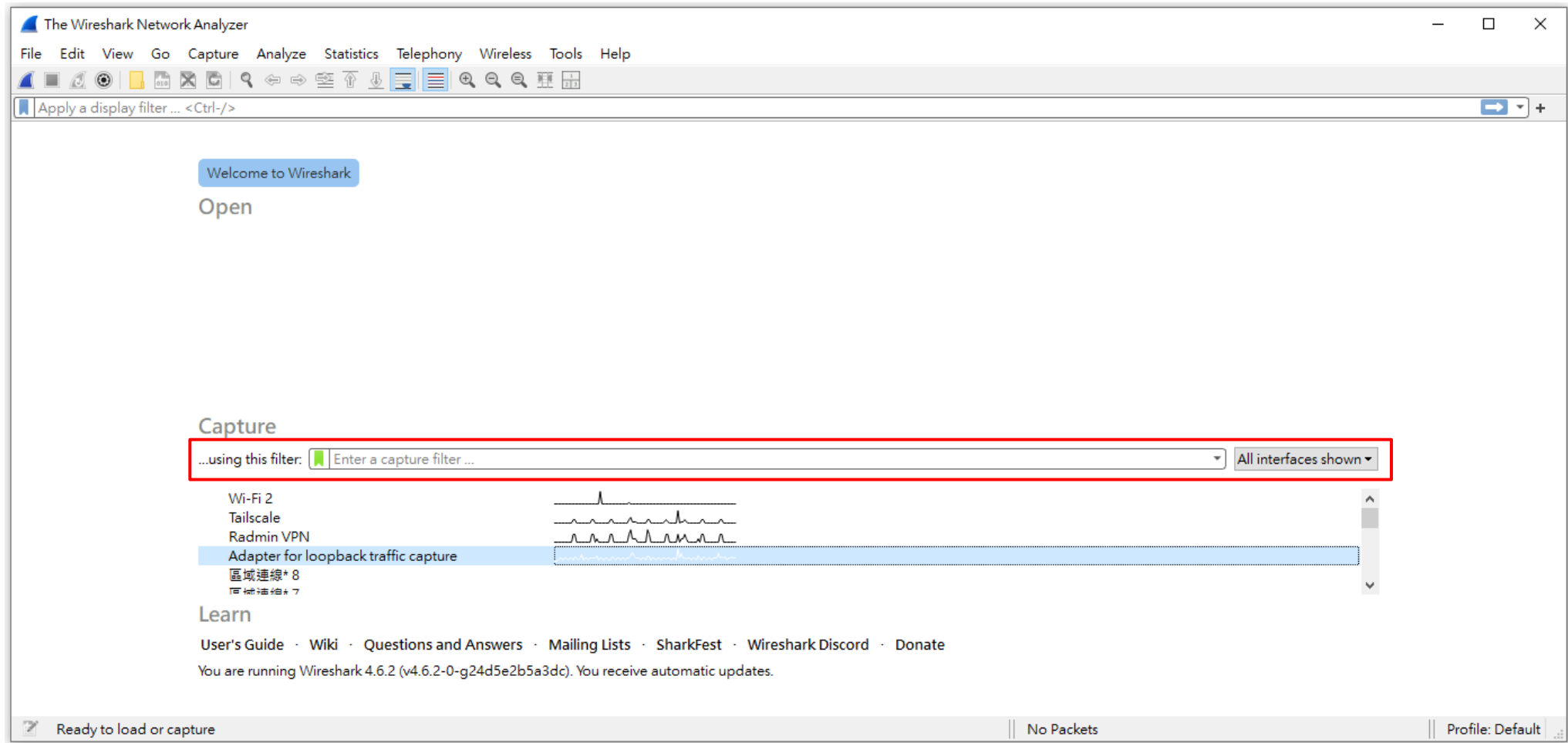
☒ Enable promiscuous mode on all interfaces
 ☐ Enable monitor mode on all 802.11 interfaces
 Manage Interfaces...

Capture filter for selected interfaces: Enter a capture filter ...
 Compile BPFs

Start
關閉
說明

Network Interface Card (NIC)

- Capture filter(擷取篩選器)



Capture filter

- Type: host, port
- Direction: src, dst
- Protocol: ip, tcp, udp, http, ftp
- Logical Operators:
 - && (AND)
 - || (OR)
 - ! (NOT)
- For Example:

Capture

...using this filter:



All interfaces shown ▼

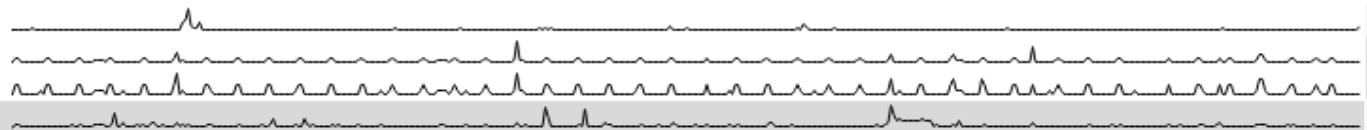
Wi-Fi 2

Tailscale

Radmin VPN

Adapter for loopback traffic capture

區域網路卡



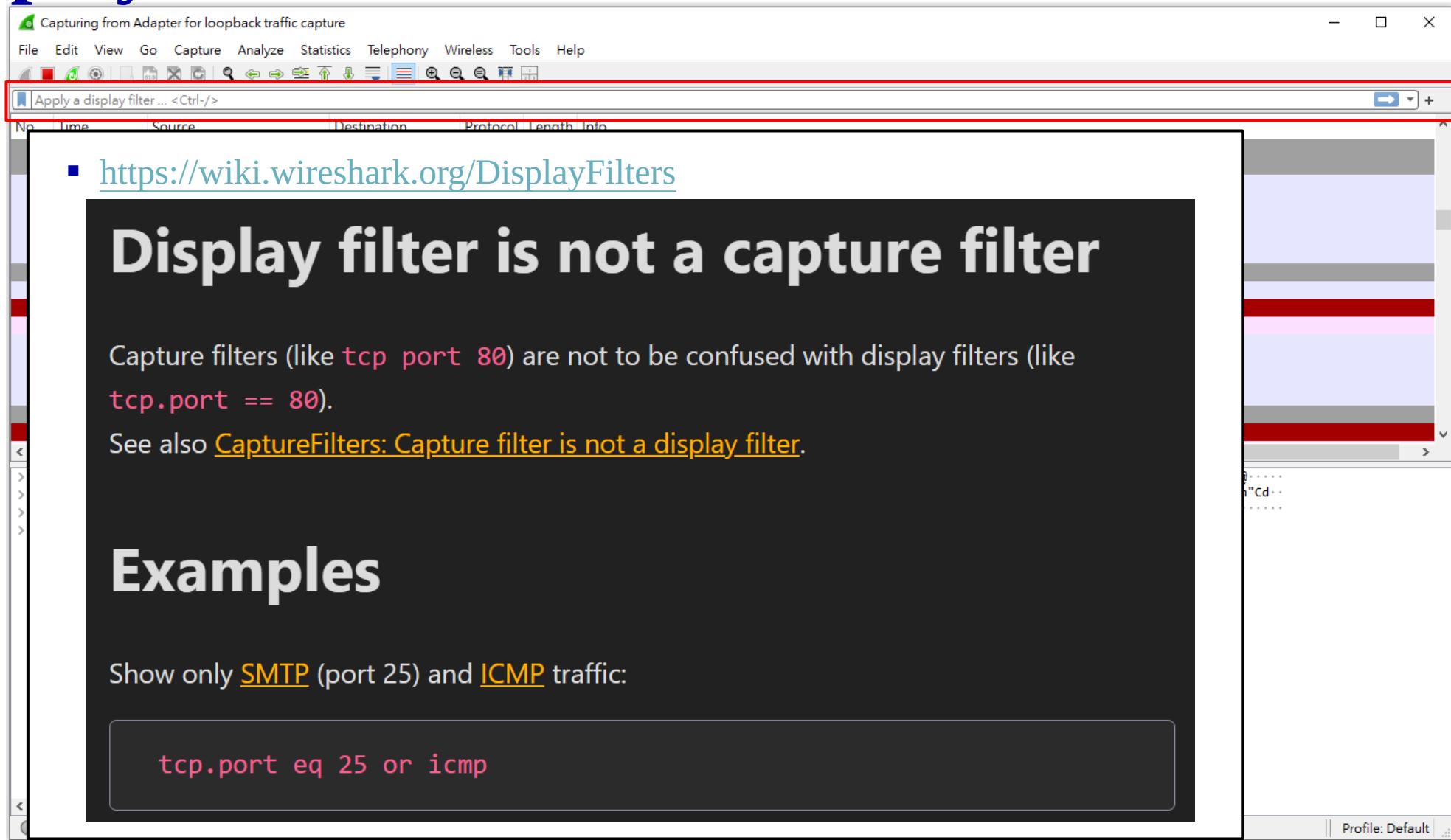
User Interface

The screenshot displays the Wireshark interface with a live capture in progress on the 'Adapter for loopback traffic capture'. The interface is divided into three main panes:

- Packet List:** A table showing captured packets. The selected packet is #48, a TCP RST, ACK from 127.0.0.1 to 127.0.0.1.
- Packet Details:** A hierarchical view of the selected packet's structure. It shows the Null/Loopback source, Internet Protocol Version 4 details (Src: 127.0.0.1, Dst: 127.0.0.1), and Transmission Control Protocol details (Src Port: 62396, Dst Port: 28194, Seq: 0, Len: 0).
- Packet Bytes:** A hex dump and ASCII representation of the packet data. The data is shown in hexadecimal and its corresponding ASCII characters.

At the bottom of the interface, it indicates 'Packets: 150' and 'Profile: Default'.

Display filter



The image shows the Wireshark network protocol analyzer interface. The title bar reads "Capturing from Adapter for loopback traffic capture". The menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. The toolbar contains various icons for file operations, capture control, and analysis. A red rectangle highlights the "Apply a display filter ..." text box in the toolbar. Below this, a large semi-transparent overlay window is displayed. The overlay has a title bar with a red square icon and a link to <https://wiki.wireshark.org/DisplayFilters>. The main content of the overlay is on a dark background and contains the following text:

- <https://wiki.wireshark.org/DisplayFilters>
- Display filter is not a capture filter**
- Capture filters (like `tcp port 80`) are not to be confused with display filters (like `tcp.port == 80`).
- See also [CaptureFilters: Capture filter is not a display filter](#).
- Examples**
- Show only **SMTP** (port 25) and **ICMP** traffic:
- `tcp.port eq 25 or icmp`

The background Wireshark interface shows a packet list on the left and a packet details pane on the right. The status bar at the bottom right indicates "Profile: Default".

Example app: UDP server

Python UDPServer

	from socket import *
	serverPort = 12000
create UDP socket	→ serverSocket = socket(AF_INET, SOCK_DGRAM)
bind socket to local port number 12000	→ serverSocket.bind(("", serverPort))
	print("The server is ready to receive")
loop forever	→ while True:
Read from UDP socket into message, getting client's address (client IP and port)	→ message, clientAddress = serverSocket.recvfrom(2048)
	modifiedMessage = message.decode().upper()
send upper case string back to this client	→ serverSocket.sendto(modifiedMessage.encode(), clientAddress)

Example app: UDP client

Python UDPClient

include Python's socket library

→ `from socket import *`

`serverName = '127.0.0.1' #hostname`

`serverPort = 12000`

create UDP socket for server

→ `clientSocket = socket(AF_INET,
SOCK_DGRAM)`

get user keyboard input

→ `message = input('Input lowercase sentence:')`

attach server name, port to message; send into socket

→ `clientSocket.sendto(message.encode(),
(serverName, serverPort))`

read reply characters from socket into string

→ `modifiedMessage, serverAddress =
clientSocket.recvfrom(2048)`

print out received string and close socket

→ `print modifiedMessage.decode()
clientSocket.close()`

UDP example in Wireshark

Windows PowerShell

```
Windows PowerShell
Copyright (C) Microsoft Corporation. 著作權所有，並保留一切權利。

請嘗試新的跨平台 PowerShell https://aka.ms/pscore6

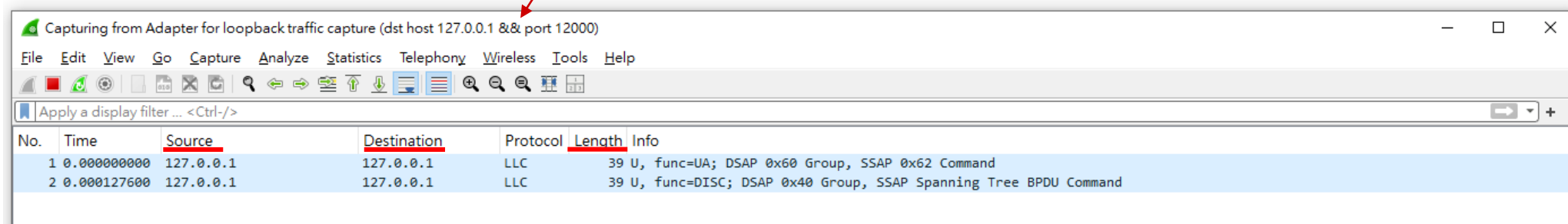
PS C:\Users\Sam Chiu> cd H:\我的雲端硬碟\114_2_computer_network\socket_py\lecuture
PS H:\我的雲端硬碟\114_2_computer_network\socket_py\lecuture> python .\UDPServer.py
The server is ready to receive
```

Windows PowerShell

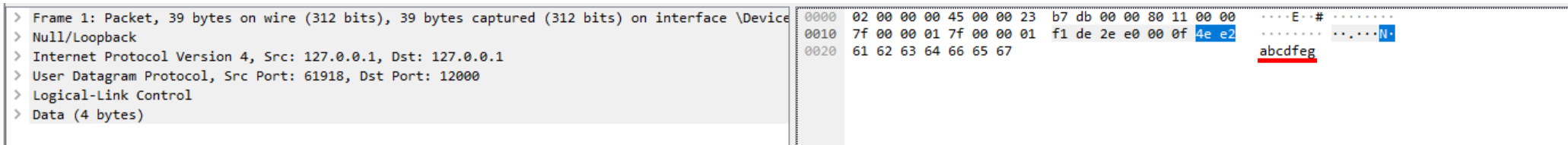
```
PS H:\我的雲端硬碟\114_2_computer_network\socket_py\lecuture> python .\UDPCClient.py
Input lowercase sentence:abcdefg
ABCDEFGG
PS H:\我的雲端硬碟\114_2_computer_network\socket_py\lecuture>
```

UDP example in Wireshark

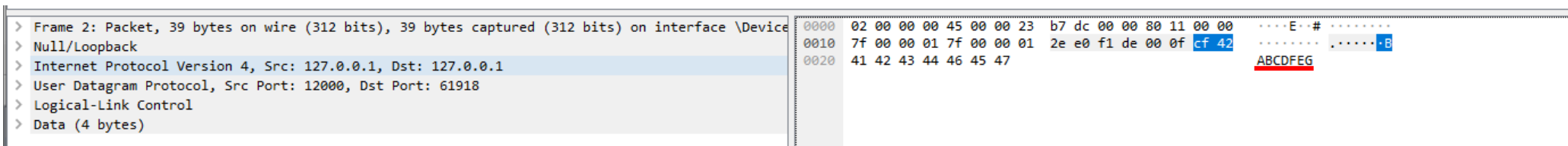
Capture filter for selected interfaces: `dst host 127.0.0.1 && port 12000`



No. 1 (Client → Server)



No. 2 (Server → Client)

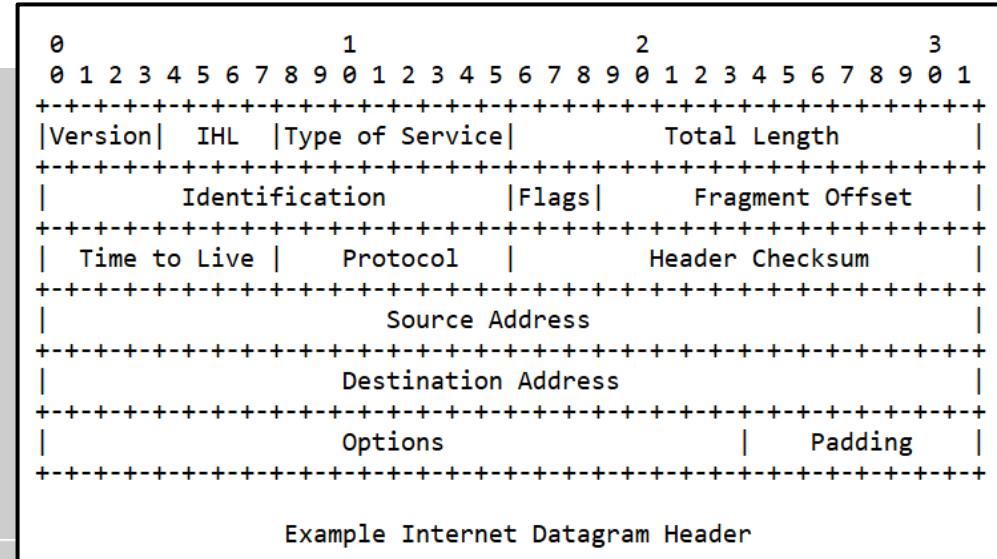


UDP example in Wireshark

RFC 791

■ Client → Server

- Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
 - 0100 = Version: 4
 - 0101 = Header Length: 20 bytes (5)
 - > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
 - Total Length: 35
 - Identification: 0xb7db (47067)
 - > 000. = Flags: 0x0
 - ...0 0000 0000 0000 = Fragment Offset: 0
 - Time to Live: 128
 - Protocol: UDP (17)
 - Header Checksum: 0x0000 [validation disabled]
 - [Header checksum status: Unverified]
 - Source Address: 127.0.0.1
 - Destination Address: 127.0.0.1
 - [Stream index: 0]



- > Frame 1: Packet, 39 bytes on wire (312 bits), 39 bytes captured
- > Null/Loopback
- > Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
- ▼ User Datagram Protocol, Src Port: 61918, Dst Port: 12000
 - Source Port: 61918
 - Destination Port: 12000
 - Length: 15
 - Checksum: 0x4ee2 [unverified]
 - [Checksum Status: Unverified]
 - [Stream index: 0]
 - [Stream Packet Number: 1]
 - > [Timestamps]
 - UDP payload (7 bytes)
 - > Logical-Link Control
 - > Data (4 bytes)

```

0000  02 00 00 00 45 00 00 23 b7 db 00 00 80 11 00 00  ....E..#
0010  7f 00 00 01 7f 00 00 01 f1 de 2e e0 00 0f 4e e2  ....N.
0020  61 62 63 64 66 65 67  ....abcdfe

```

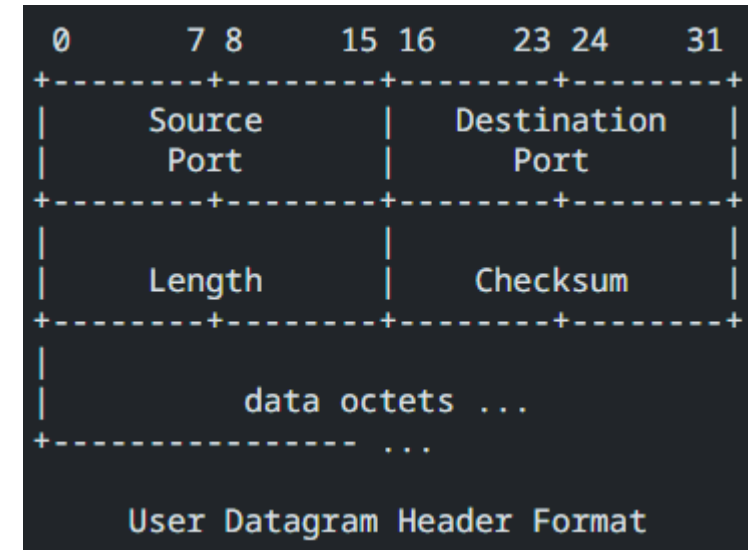
UDP example in Wireshark

■ Client → Server

```

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 35
    Identification: 0xb7db (47067)
  > 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 128
    Protocol: UDP (17)
    Header Checksum: 0x0000 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 127.0.0.1
    Destination Address: 127.0.0.1
    [Stream index: 0]

```



```

<
> Frame 1: Packet, 39 bytes on wire (312 bits), 39 bytes captured
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
  > User Datagram Protocol, Src Port: 61918, Dst Port: 12000
    < Source Port: 61918
    < Destination Port: 12000
    < Length: 15
    < Checksum: 0x4ee2 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 0]
    [Stream Packet Number: 1]
  > [Timestamps]
    < UDP payload (7 bytes)
  > Logical-Link Control
  > Data (4 bytes)

```

```

0000 02 00 00 00 45 00 00 23 b7 db 00 00 80 11 00 00 ....E..#
0010 7f 00 00 01 7f 00 00 01 f1 de 2e e0 00 0f 4e e2 .....N.
0020 61 62 63 64 66 65 67 abcdfeg

```

UDP example in Wireshark

- Server → Client

Capturing from Adapter for loopback traffic capture (dst host 127.0.0.1 && port 12000)

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

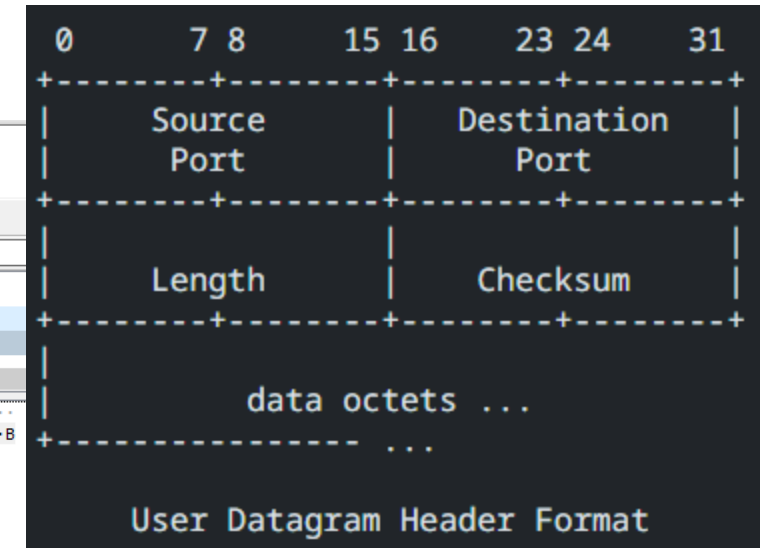
Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	LLC	39	U, func=UA; DSAP 0x60 Group, SSAP 0x62 Command
2	0.000127600	127.0.0.1	127.0.0.1	LLC	39	U, func=DISC; DSAP 0x40 Group, SSAP Spanning Tree BPDU Command

<

> Frame 2: Packet, 39 bytes on wire (312 bits), 39 bytes captured
 > Null/Loopback
 > Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
 > User Datagram Protocol, Src Port: 12000, Dst Port: 61918
 Source Port: 12000
 Destination Port: 61918
 Length: 15
 Checksum: 0xcf42 [unverified]
 [Checksum Status: Unverified]
 [Stream index: 0]
 [Stream Packet Number: 2]
 > [Timestamps]
 UDP payload (7 bytes)
 > Logical-Link Control
 > Data (4 bytes)

0000 02 00 00 00 45 00 00 23 b7 dc 00 00 80 11 00 00 ... E..#
 0010 7f 00 00 01 7f 00 00 01 2e e0 f1 de 00 0f cf 42B
 0020 41 42 43 44 46 45 47 ABCDFEG



Classwork 3

- Please draw the **FULL** Sequence Timing Diagram of the UDP Example



Time

A vertical blue line with a downward-pointing arrowhead at the bottom, indicating the progression of time.

Homework 1

- TCP Client/Server Programming & Packet Analysis

1. Implement a TCP Client/Server program with Functionalities:

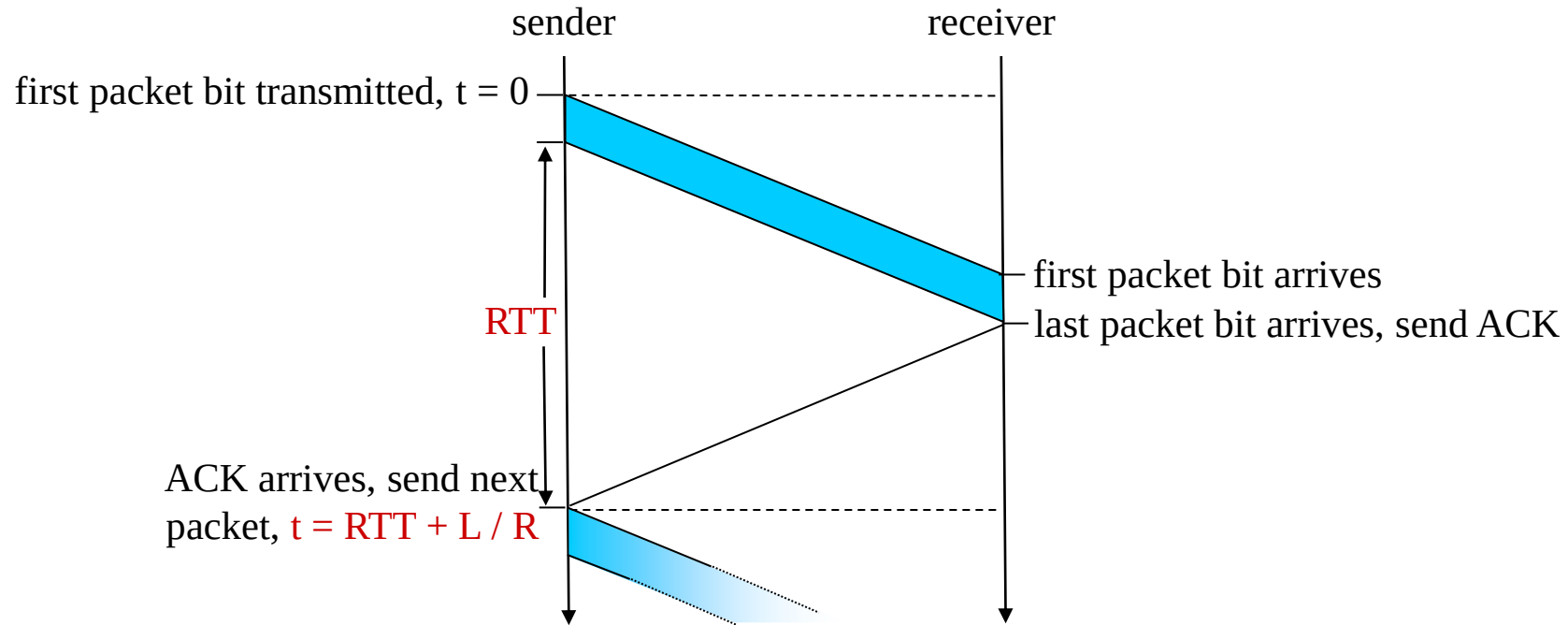
- Client:
 - (1) Continuously input two numbers (comma-separated) and send them to Server.
 - (2) Input 'exit' to close the connection.
- Server: Receive numbers, compute the product, and send back to Client.

2. Using Wireshark to capture and analyze the entire communication process. provide a report including screenshots, labels, and descriptions for the following:

- A) TCP three-way handshake packets
- B) First Application Data Packet
- C) **Application-level** RTT (Round Trip Time)
- D) Connection Close Packets
- E) Time Sequence Diagram(including seq# and ack#)

RTT (Round Trip Time)

RTT: Client sent $\rightarrow$ Client received



RTT: Client sent → Client received

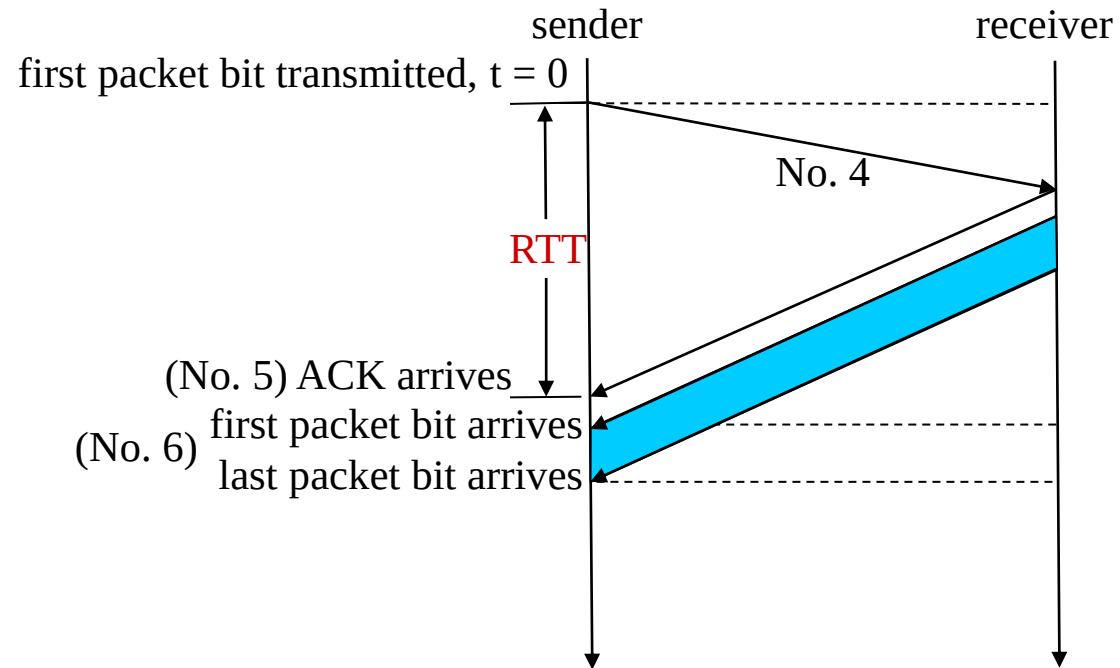
Capturing from Adapter for loopback traffic capture (dst host 127.0.0.1 && port 12000)

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

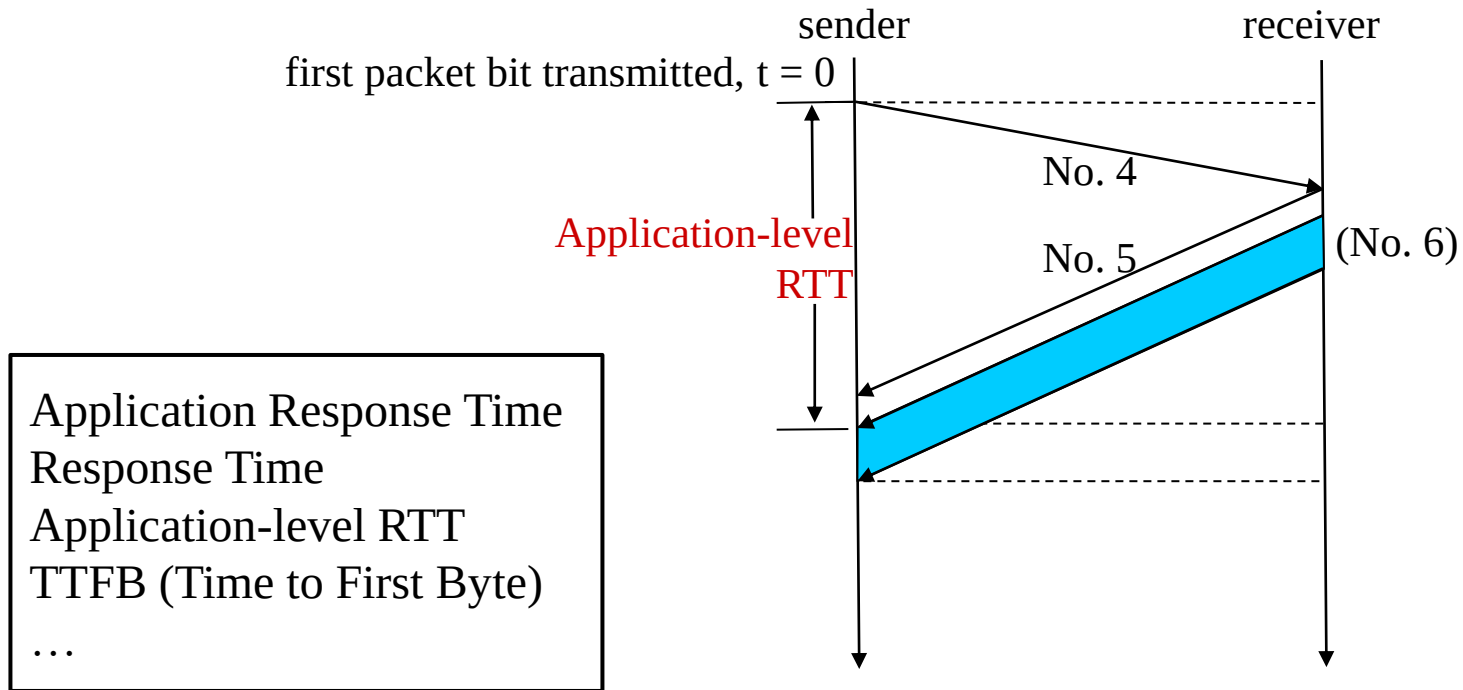
Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	56	64561 → 12000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
2	0.000097700	127.0.0.1	127.0.0.1	TCP	56	12000 → 64561 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
3	0.000127500	127.0.0.1	127.0.0.1	TCP	44	64561 → 12000 [ACK] Seq=1 Ack=1 Win=2619648 Len=0
4	3.913081000	127.0.0.1	127.0.0.1	TCP	49	64561 → 12000 [PSH, ACK] Seq=1 Ack=1 Win=2619648 Len=5
5	3.913117100	127.0.0.1	127.0.0.1	TCP	44	12000 → 64561 [ACK] Seq=1 Ack=6 Win=2619648 Len=0
6	3.913175400	127.0.0.1	127.0.0.1	TCP	50	12000 → 64561 [PSH, ACK] Seq=1 Ack=6 Win=2619648 Len=6
7	3.913193400	127.0.0.1	127.0.0.1	TCP	44	64561 → 12000 [ACK] Seq=6 Ack=7 Win=2619648 Len=0
8	7.537040800	127.0.0.1	127.0.0.1	TCP	49	64561 → 12000 [PSH, ACK] Seq=6 Ack=7 Win=2619648 Len=5
9	7.537081100	127.0.0.1	127.0.0.1	TCP	44	12000 → 64561 [ACK] Seq=7 Ack=11 Win=2619648 Len=0
10	7.537129900	127.0.0.1	127.0.0.1	TCP	50	12000 → 64561 [PSH, ACK] Seq=7 Ack=11 Win=2619648 Len=6
11	7.537148400	127.0.0.1	127.0.0.1	TCP	44	64561 → 12000 [ACK] Seq=11 Ack=13 Win=2619648 Len=0
12	8.982181300	127.0.0.1	127.0.0.1	TCP	48	64561 → 12000 [PSH, ACK] Seq=11 Ack=13 Win=2619648 Len=4
13	8.982217700	127.0.0.1	127.0.0.1	TCP	44	12000 → 64561 [ACK] Seq=13 Ack=15 Win=2619648 Len=0
14	8.982356900	127.0.0.1	127.0.0.1	TCP	44	12000 → 64561 [FIN, ACK] Seq=13 Ack=15 Win=2619648 Len=0
15	8.982378200	127.0.0.1	127.0.0.1	TCP	44	64561 → 12000 [ACK] Seq=15 Ack=14 Win=2619648 Len=0
16	8.983645000	127.0.0.1	127.0.0.1	TCP	44	64561 → 12000 [FIN, ACK] Seq=15 Ack=14 Win=2619648 Len=0
17	8.983686800	127.0.0.1	127.0.0.1	TCP	44	12000 → 64561 [ACK] Seq=14 Ack=16 Win=2619648 Len=0

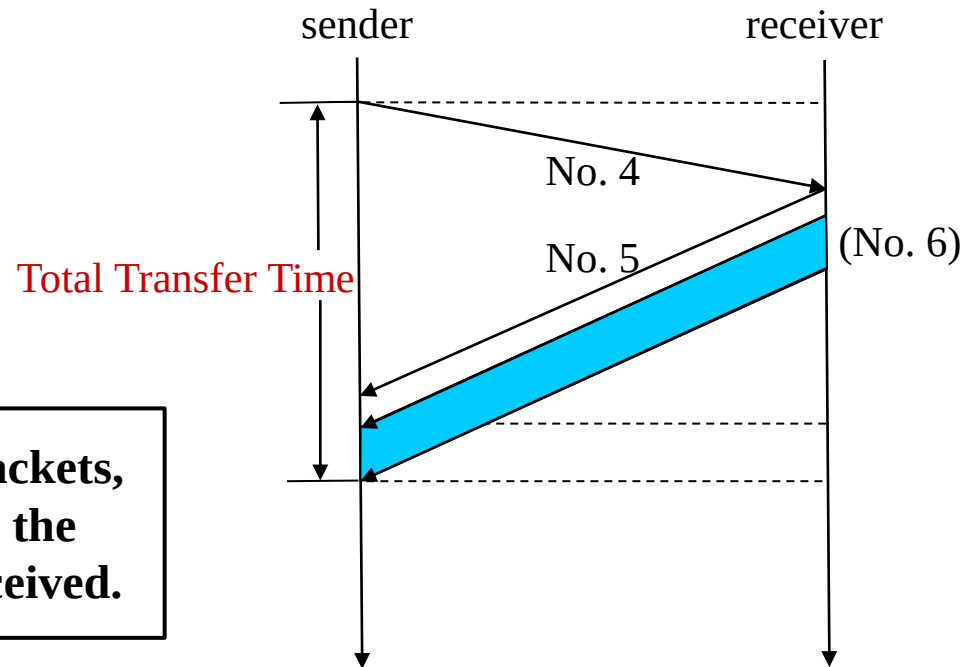
Network Round Trip Time (SampleRTT)



Application Response Time (Application-level RTT)



Total Transfer Time (Completion Time)



If there are multiple packets,
calculate the time until the
'last' packet is fully received.

Submission

Include the following in your report:

- Your name and ID.
- Client & Server source code.
- Wireshark screenshots with labels and descriptions for following:
 - A) TCP three-way handshake packets
 - B) First Application Data Packet
 - C) RTT (Round Trip Time)
 - D) Connection Close Packets
 - E) Time Sequence Diagram

Report Format

- The report should be prepared in **slide format** (e.g., PowerPoint or similar).
- Export as a **PDF file** with the file name: **[ID]_hw1.pdf** before submission.

Deadline: 2 weeks